Introduction

If you've written Python for more than a few weeks, you've used `with open(...) as f:` a hundred times without asking what it actually does. That's fine — until the day you need to write your own version. Maybe you're managing a database connection that has to close even if a query fails. Maybe you're holding a lock that absolutely cannot stay locked after an exception. Maybe you're timing a block of code and want the timer to stop no matter what happens inside it.

At that point, "I use `with` for files" isn't enough. You need to understand the actual protocol behind it — two methods, `__enter__` and `__exit__`, and the guarantee Python makes about when they run.

This also happens to be a favorite interview topic, not because it's obscure, but because it separates people who've memorized `with open(...)` from people who understand *why* it's safe. Interviewers ask about context managers to see if you understand resource management, exception propagation, and the difference between "code that usually cleans up" and "code that is guaranteed to clean up."

By the end of this lesson you'll be able to write a context manager two different ways — as a class, and as a four-line generator function — and you'll know exactly which one to reach for.

Problem Overview

Here's the situation that breaks people the first time they hit it: you open a resource (a file, a connection, a lock), do some work with it, and then something goes wrong — an exception gets raised. If your cleanup code (like `file.close()`) is written as the last line of a function, and the exception happens before that line, cleanup never runs. The exception jumps straight over it.

In a short-lived script, you might never notice. The process exits and the OS reclaims everything anyway. But in a long-running service, this is how you leak file handles, leave stale locks in place, or hold onto database connections until the pool runs dry.

The fix people reach for first is `try/finally`. That works, but it's easy to forget, verbose to repeat everywhere, and easy to get subtly wrong (what exactly do you do with the exception info in `finally`?). Python's answer is the context manager protocol: an object that defines what "setup" and "teardown" mean, paired with the `with` statement, which guarantees teardown runs no matter how the block exits — normal return, early `return`, or exception.

Example

Take a broken version first:

```
f = open("log.txt", "w")
f.write("starting job\n")
raise ValueError("something went wrong")
f.close()  # never runs
```

The file is left open. Python doesn't get a chance to skip a line politely — the exception simply propagates past `f.close()` because that line is never reached.

Now the same logic with `with`:

```
with open("log.txt", "w") as f:
    f.write("starting job\n")
    raise ValueError("something went wrong")
```

The exception still gets raised — `with` doesn't hide errors — but the file is closed before the exception continues upward. You didn't write `try`, you didn't write `finally`, you didn't call `close()`. The `with` statement did it for you.

That's the entire value proposition: **cleanup is guaranteed, regardless of how the block ends.**

Intuition

Think about a hotel key card. Pull it from the slot by the door and the lights cut immediately — it doesn't matter if you left calmly through the front door or climbed out a window mid-emergency. The room doesn't care *how* you left. It only cares that when you're gone, the lights go off.

That's the guarantee a context manager gives you for code. It doesn't matter whether your block finishes normally or blows up with an exception three lines in — the teardown step runs regardless.

Here's how an experienced engineer arrives at this design: they notice that "setup, do work, guaranteed teardown" is a pattern that shows up constantly — opening and closing files, acquiring and releasing locks, connecting and disconnecting from a database, starting and stopping a timer. Rather than writing `try/finally` by hand every single time, Python lets you define the setup and teardown once, as two methods, and then hands control to `with` to guarantee the teardown always fires.

The two methods are:

- `__enter__(self)` — runs when the `with` block starts. Whatever it returns becomes the `as` variable.
- `__exit__(self, exc_type, exc_value, traceback)` — runs when the block ends, for *any* reason. If an exception occurred, Python hands it to `__exit__` as three arguments instead of skipping the call.

The detail that trips people up: if `__exit__` returns `True`, Python treats the exception as handled and swallows it — silently. Return `False` or `None` (the default), and the exception keeps traveling upward like normal. Returning `True` out of habit, without meaning to suppress anything, is a classic way to lose an hour wondering why an error you know is happening seems to vanish.

Brute Force Solution

"Brute force" here just means: write the class version by hand.

Idea: define a class with `__enter__` and `__exit__`. `__enter__` does setup and returns whatever the `with` block should bind to. `__exit__` does teardown and returns `False` so exceptions still propagate.

```
import time

class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        elapsed = time.perf_counter() - self.start
        print(f"Elapsed: {elapsed:.4f}s")
        return False
```

Algorithm: 1. Python sees `with Timer() as t:` and calls `Timer().__enter__()`. 2. The return value of `__enter__` is bound to `t`. 3. Everything inside the `with` block runs. 4. The instant the block ends — for any reason — `__exit__` runs with exception info if there was one.

Advantages: explicit, easy to add state (multiple attributes, helper methods), easy to extend with more behavior later.

Disadvantages: verbose for something this small — a whole class just to time a block of code.

Time complexity: O(1) overhead for the protocol itself — the cost is whatever your setup/teardown code does. **Space complexity:** O(1) beyond whatever state you choose to store on `self`.

Optimal Solution

For a one-off case like the timer, writing a full class is more ceremony than the problem needs. Python's `contextlib.contextmanager` decorator lets you write the same enter/exit behavior as a single generator function with one `yield` in the middle.

The trick: **everything before `yield` is `__enter__`. Everything after `yield` is `__exit__`.** Python pauses execution right at the `yield`, hands control to your `with` block, and resumes after `yield` (wrapped in an implicit `try/finally`) once the block ends.

Class version	Generator version
`__enter__` body	code before `yield`
value returned by `__enter__`	value passed to `yield`
`__exit__` body	code after `yield`
`return False` at the end of `__exit__`	do nothing — exceptions propagate automatically unless caught

Python Code

```
import time
from contextlib import contextmanager


@contextmanager
def timer():
    start = time.perf_counter()
    yield
    elapsed = time.perf_counter() - start
    print(f"Elapsed: {elapsed:.4f}s")
```

That's the whole thing — four lines instead of a class.

Code Walkthrough

- `@contextmanager` wraps the generator function so it behaves like a context manager when used with `with`.
- `start = time.perf_counter()` runs when `with timer():` starts — this is the `__enter__` half.
- `yield` is the handoff point. Python pauses here and lets the code inside the `with` block execute. If you write `yield value`, that `value` is what gets bound by `as`.

- The lines after `yield` are the `__exit__` half — they run once the `with` block finishes, whether it finished normally or an exception was raised inside it. `contextmanager` wraps this in a `try/finally` for you, so the teardown code still runs even on an exception, and the exception still propagates afterward unless you explicitly catch and suppress it.

Dry Run

```
with timer():
    total = sum(range(10_000_000))
```

1. `timer()` is called, creating a generator.
2. `with` calls the equivalent of `__enter__`, which runs `start = time.perf_counter()` and then hits `yield`.
3. Execution pauses at `yield` and control passes to the `with` block.
4. `total = sum(range(10_000_000))` runs.
5. The block ends. Python resumes the generator right after `yield`.
6. `elapsed = time.perf_counter() - start` and `print(...)` run, printing something like `Elapsed: 0.3120s`.

Now with an exception:

```
with timer():
    raise ValueError("boom")
```

Steps 1–3 are identical. Then the `raise` fires inside the block. Because `contextmanager` wraps the post-`yield` code in `try/finally`, the `elapsed` / `print` lines still run — the timer still reports — and then `ValueError("boom")` continues propagating upward exactly as if `with` weren't there at all.

Complexity Analysis

Time: O(1) overhead from the protocol itself in both the class and generator versions — one function call in, one function call out. The actual cost is dominated by whatever setup/teardown work you do (opening a file, acquiring a lock, etc.), not by the protocol.

Space: O(1) — a generator context manager keeps only its local frame alive between `yield` calls, and a class context manager only holds whatever attributes you assign to `self`. Neither approach scales with the size of the work done inside the `with` block.

Both are correct because the guarantee doesn't come from complexity — it comes from Python's exception-propagation mechanics: `__exit__` (or the code after `yield`) is called unconditionally whenever the block exits, regardless of *why* it exits.

Alternative Solutions

The realistic alternative is plain `try/finally`:

```
f = open("log.txt", "w")
try:
    f.write("starting job\n")
    raise ValueError("boom")
finally:
    f.close()
```

This works and needs no extra machinery. But it doesn't scale — every caller has to remember to wrap their code correctly, and there's no reusable, named unit of "how do I set up and tear down this resource." Context managers let you write the setup/teardown logic exactly once and reuse it anywhere with `with resource():`. For a one-off, throwaway block, `try/finally` is fine. For anything you'll use more than once, a context manager is the better investment.

Edge Cases

- **Empty block:** `with timer(): pass` — still runs enter and exit correctly; elapsed time will just be near-zero.
- **Multiple exceptions in nested `with` blocks:** each context manager's `__exit__` runs in reverse order of entry, and each gets a chance to see the exception.
- **`__exit__` returning `True` by mistake:** silently swallows the exception — a subtle bug that looks like the error "just disappeared."
- **Generator-based context manager with more than one `yield`:** `contextlib.contextmanager` raises a `RuntimeError` — it strictly expects exactly one `yield`.
- **Exception raised inside `__enter__` itself:** `__exit__` is never called, because the block never actually started.
- **Reusing a generator-based context manager instance twice:** generators are single-use; calling the decorated function again creates a fresh one, but reusing the *same* generator object with two `with` statements will fail.

Common Mistakes

1. Forgetting `return False` (or omitting a return statement) — usually harmless since `None` is falsy, but explicit is safer, especially if a teammate later "fixes" it to `return True` without knowing the consequence.
2. Returning `True` from `__exit__` believing it means "exit cleanly" when it actually means "suppress the exception."
3. Writing more than one `yield` in a `contextmanager`-decorated generator.
4. Doing setup work *after* `yield` instead of before it — reversing enter and exit responsibilities.
5. Forgetting that `contextlib.contextmanager` wraps the teardown in `try/finally` for you — manually adding another `try/finally` around the `yield` is redundant.
6. Assuming `with` catches and hides all exceptions by default — it only does that if `__exit__` returns `True`.
7. Using a class-based context manager for simple, stateless setup/teardown when a four-line generator would be clearer.

Interview Questions

- What are the two dunder methods behind the `with` statement, and what does each receive/return?
- What happens if `__exit__` returns `True`? Why is that dangerous?
- How would you implement a context manager using `contextlib.contextmanager` instead of a class?
- What happens if an exception is raised inside `__enter__`?
- Can you write a context manager that retries the block on failure? (Hint: think about what `__exit__` can inspect and do with `exc_type`.)
- What's the difference between `with` and `async with`?
- Why can't a generator-based context manager be reused across two separate `with` statements?

Similar Problems

- **Implementing a custom iterator (`__iter__` / `__next__`)** — same "protocol via dunder methods" pattern as context managers.
- **Writing a decorator with `functools.wraps`** — another case where Python gives you a small protocol to hook into standard behavior.

- **Thread lock usage (`threading.Lock` as a context manager)** — a real-world case of the exact enter/exit pattern from this lesson.
- **Database connection pooling with `with conn:`** — same setup/teardown guarantee applied to a network resource instead of a file.
- **`contextlib.suppress`** — a built-in context manager worth reading the source of once you understand `__exit__`'s return value.

Key Takeaways

- `with` guarantees cleanup code runs no matter how a block exits — normal completion, early return, or exception.
- The protocol is just two methods: `__enter__` for setup, `__exit__` for teardown, with `__exit__` receiving exception details if one occurred.
- Returning `True` from `__exit__` swallows the exception — almost always a mistake unless you mean to suppress it.
- `contextlib.contextmanager` turns the same protocol into a single generator function: everything before `yield` is enter, everything after is exit.
- Reach for a class when you need real state and structure; reach for `contextmanager` when a function with one `yield` says the same thing in fewer lines.
- The same pattern covers files, locks, database connections, and temporary setting changes — not just files.

Watch the Video

This lesson is easier to absorb watching it happen step by step — tracing exactly where an exception goes inside a `with` block, and watching the class rewritten into the generator version live. Watch it here: {LINK}

About the Series

This lesson is part of *Fun with Learning Technology*, a series built around one idea: you should understand the mechanism behind a language feature well enough to build it yourself, not just memorize the syntax that uses it. Each lesson takes something you've probably used without thinking about — decorators, context managers, comprehensions — and opens it up until it stops feeling like magic.

Call To Action

If this made `with` finally click, the video walkthrough makes it click even faster — give it a watch and a like on YouTube, and subscribe so the next lesson in the series lands in your feed. Prefer reading over your morning coffee? Subscribe to the Substack for the written version of every lesson. And if you've got a topic you want unpacked this same way, drop it in the comments — that's genuinely where most of these lessons come from.

Quick Review

What triggers use of a context manager?

Any setup/teardown pair that must run together — file handles, locks, db connections, temp state — signals `with`.

What two dunder methods define a context manager?

`__enter__` (setup, returns value for `as`) and `__exit__` (teardown, receives exception info if raised).

What does `__exit__` returning True do?

Suppresses the exception — it won't propagate. Returning False/None lets it re-raise normally.

Common bug when hand-writing __exit__?

Forgetting to check exception args before cleanup, or accidentally swallowing exceptions by returning truthy from __exit__.

contextlib.contextmanager vs class-based context manager?

Decorator wraps a generator: code before `yield` is __enter__, after is __exit__; less boilerplate than writing a class.

Interview follow-up: how do exceptions inside a `with` block reach __exit__?

Python calls __exit__ with (exc_type, exc_value, traceback) before propagating; __exit__ can inspect or suppress it.

Python Lesson 25: Context Managers (The with protocol, __enter__/__exit__, and contextlib.contextmanager) — free Python cheat sheet